

AI Career Roadmap Generator v1

AI Testing Report

Prepared by: Mahnoor Sohail

Project: Hiresense AI - Week 4, Module 2

1. Overview

This report covers the AI Career Roadmap Generator v1 - a system that uses a real LLM (Google Gemini) to generate a personalized career roadmap from a candidate's actual detected skill gaps, then attaches real learning resources to it. This report focuses specifically on how the AI portion was designed, tested with real API calls (not mock responses), and evaluated for hallucination.

Task	Deliverable	Status
1. Design the AI Output	module2_task1_design.md	Complete
2. Prompt Engineering (A/B/C/D)	prompts/	Complete
3. Structured JSON Schema	roadmap_schema.json	Complete
4. Personalized Roadmap	roadmap_generator.py	Complete
5. Resource Recommendation	recommendation_engine.py	Complete
6. Hallucination Testing	llm_evaluation.csv	Complete

2. System Design

Input / Output

- **Input:** Candidate Profile + ATS Analysis + Missing Skills + Career Goal.
- **Output:** career_summary, current_level, skill_gaps, roadmap, recommended_resources, resume_improvements - a fixed JSON shape, validated against a schema, not free text.
- Skill gaps and their priority (critical / nice-to-have) are inherited directly from Week 3's already-verified engine - the AI's job is to explain and sequence facts that were already computed correctly, not to re-derive them.

The Four Prompts

- **Prompt A - Career Analysis:** determines current level, strongest/weak areas, career direction.
- **Prompt B - Skill Gap Explanation:** explains why each missing skill matters, batched into one call per roadmap (not one call per skill) to conserve free-tier API quota.
- **Prompt C - Roadmap Generation:** sequences skills into 5 fixed phases (Fundamentals, Intermediate Skills, Advanced Skills, Projects, Job Preparation). Explicitly forbidden from recommending specific courses or links.
- **Prompt D - Resume Improvements:** not one of the originally named prompts, but added because the required output schema included a Resume Improvements field with no prompt assigned to generate it. Added rather than leaving that field hardcoded empty.

Resource Safety Design

The single most important design decision in this system: resources are NEVER generated by the LLM. Prompt C is explicitly told not to recommend courses or links - it only names which skills belong in which phase. Real

resources are attached afterward by matching against a verified database of 24 real, stable sources (official docs, Kaggle Learn, Khan Academy, etc.). This makes a hallucinated course name or broken link structurally impossible for this part of the output, rather than merely unlikely.

3. Testing Methodology

All testing in this report used the real Gemini API (gemini-3.5-flash-lite), not mock responses. A separate mock mode exists in the code to test the surrounding assembly logic without API access, but every result in this report reflects genuine model output.

Three test cases were designed to probe specific hallucination risks named in the task brief:

- **Test 1 - Invented skill name:** a deliberately fake skill ("Quantum Flux Programming", not a real technology) was included in the skill gap list, to see whether the model would recognize it as unfamiliar or confidently fabricate an explanation for it.
- **Test 2 - Near-beginner targeting a Senior role:** a candidate with zero real skills or experience, targeting a Senior Software Engineer position, to check whether the roadmap still sequenced fundamentals before advanced topics rather than jumping ahead inappropriately.
- **Test 3 - Normal case, skill leakage check:** a realistic candidate and skill gap list, checking specifically whether every skill placed in the roadmap actually came from the original list given, with none invented.

Automated checks used

- **no_invented_skills:** every skill in the roadmap traces back to the original skill_gaps list.
- **all_resources_verified:** every recommended resource URL traces back to the verified database.
- **phase_structure_correct:** exactly 5 phases, correct names, correct order.
- **no_confident_fabrication_for_fake_skill** (Test 1 only): keyword-based check for whether the model admitted uncertainty about the fake skill - flagged as a starting signal only; the full text still requires a human to read, since a model could pass a keyword check while still subtly fabricating elsewhere.

4. Results

Structural checks: all passed, across all 3 test cases

- No invented skills in any roadmap.
- All recommended resource URLs verified against the database.
- Correct 5-phase structure and order in every test case.

The real finding: a genuine hallucination (Test 1, before fix)

Given the fake skill "Quantum Flux Programming," the model did not flag it as unrecognized. Instead it confidently produced a technical-sounding justification, citing fabricated concepts including "sub-atomic data processing" and "theoretical compute speeds" as reasons the skill mattered for a Software Engineer role. This is a confirmed hallucination: the model invented a plausible-sounding but entirely false technical explanation rather than admitting it did not recognize the term.

The fix

- Updated Prompt B's system instructions to explicitly require the model to state when it does not confidently recognize a skill as an established technology, rather than inventing an explanation.

Re-test result (after fix)

Same test case, same fake skill. The model's response changed to: "I don't recognize this as an established skill/technology." The hallucination did not recur. All other test cases (2 and 3) continued to produce specific, accurate, well-grounded explanations for real skills, both before and after this change.

Honest limitation

This is a mitigation, not a guarantee. A differently-phrased fake skill, a different model, or a more subtle false premise could still bypass this instruction. Prompt-level instructions reduce hallucination risk; they do not eliminate it structurally the way the resource-verification design does.

5. Conclusion and Recommendations

The system's structural safeguards (skill grounding, verified-only resources, fixed phase schema) held up completely across all three tests. The one confirmed hallucination occurred in free-text explanation generation, not in the structured/verified parts of the output - and was measurably reduced by a direct prompt instruction. Before a v2, I would recommend:

- Testing a wider variety of fake and edge-case skill names, not just one, to see how consistently the fix holds.
- Testing hallucination behavior across a second model, since this mitigation was only verified on one.
- Considering a lightweight skill-name validation step before any skill reaches the LLM at all, as a structural (not just prompt-level) safeguard for this specific failure mode.